

系统工具开发课程实验报告（第三周）

廖世嘉 25020007073
25 级计算机科学与技术

2026 年 9 月 7 日

目录

1	实验基本信息	3
2	实验目的	3
3	实验环境与工具	3
4	课上实验一：从源码构建并在干净环境安装 Wheel	3
4.1	实验要求	3
4.2	包结构与构建	3
4.3	结果	4
5	课上实验二：让编程智能体进入可验证的修复循环	4
5.1	实验要求	4
5.2	过程	4
5.3	结论	5
6	课上实验三：把”无法处理”的协作材料改成可执行信息	5
6.1	实验要求	5
6.2	重写结果	5
7	课上实验四：修复一个可复现的线性回归训练循环	5
7.1	实验要求	5
7.2	补全与结果	5
8	课后练习（第三周·共 12 个）	6
8.1	练习一：venv 环境差异——printenv 对比与 deactivate	6
8.2	练习二：创建 Python 包并生成 lockfile	6
8.3	练习三：Docker 本地构建 Missing Semester 课程网站	6
8.4	练习四：Dockerfile + docker-compose 运行应用与 Redis 缓存	7
8.5	练习五：发布包到 TestPyPI 并推送镜像到 ghcr.io	7
8.6	练习六：用 GitHub Pages 做网站	7
8.7	练习七：同一编程任务用四种模式各做一次	7
8.8	练习八：用 AI 智能体浏览陌生代码库	7

目录	2
8.9 练习九：氛围编程 (Vibe Coding) 一个小应用	7
8.10 练习十：为编程智能体创建 AGENTS.md、Skill 与子智能体	8
8.11 练习十一：Markdown 无序列表正则 (不直接编辑文件)	8
8.12 练习十二：搭建隔离环境运行 YOLO 模式智能体	8
9 GitHub 提交记录	8
10 实验结果汇总	10
11 问题与解决方法	10
11.1 本机 fake-IP DNS 与代理问题 (练习三/四/十二)	10
11.2 真实发布需要凭据 (练习五/六)	10
11.3 安全策略拦截”演示破坏性命令” (练习十二)	10
11.4 智能体直接编辑文件的局限 (练习十一)	11
12 解题感悟与实验总结	11

1 实验基本信息

表 1: 实验基本信息

项目	内容
姓名	廖世嘉
学号	25020007073
专业	25 级计算机科学与技术
实验环境	Linux (WSL) / Python / uv / Docker Compose / Git / XeLaTeX
课上实验	4 个 (构建 Wheel、智能体修复、协作改写、PyTorch 训练循环)
课后练习	12 个 (venv、打包、Docker、发布、Pages、智能体编程、隔离沙箱等)
GitHub 仓库	https://github.com/MrLLLiao/SystemToolkitDevCourse

2 实验目的

本实验围绕 Python 包打包与分发、Docker 容器化、软件发布与智能体编程两大主题展开。课上完成四个连续实验：从源码构建 wheel 并在干净环境安装、让编程智能体进入可验证的修复循环、把无法处理的协作材料改写为可执行信息、修复 PyTorch 线性回归训练循环。课后通过 12 个练习进一步掌握虚拟环境、pyproject.toml 打包、lockfile、Docker Compose 编排、TestPyPI/ghcr.io 发布、GitHub Pages 建站、四种编码模式对比、智能体浏览陌生代码库、氛围编程、AGENTS.md/Skill/子智能体、正则练习与容器隔离沙箱等内容。

3 实验环境与工具

本实验在 Windows 11 + WSL Ubuntu 26.04 环境中完成。主要工具：Python 3.14、uv (包管理)、venv (虚拟环境)、build、twine、pytest、Docker Engine + Compose v2、Git、ghcr.io / 本地 registry、GitHub Pages、pypiserver、PyTorch (CPU)、XeLaTeX/latexmk。实验中全程遵守“分步提交、禁止单次全量提交”的 Git 规范，每个练习一个独立 commit。

4 课上实验一：从源码构建并在干净环境安装 Wheel

4.1 实验要求

在 q09 中创建一个最小 Python 命令行包 greetlab，包含 src/greetlab/__init__.py、cli.py 和 pyproject.toml。运行 `python -m build` 生成 wheel，新建干净虚拟环境只从 wheel 安装，切换到 q09 之外运行 `sdt-greet -name < 学号 >` 确认输出正确。

4.2 包结构与构建

项目使用 src 布局，pyproject.toml 配置 setuptools 构建后端与入口点：

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"
```

```
[project]
name = "greetlab-25020007073"
version = "0.1.0"

[project.scripts]
sdt-greet = "greetlab.cli:main"

[tool.setuptools.packages.find]
where = ["src"]
```

使用 `python -m build --no-isolation` 生成 sdist 与 wheel，在干净 `venv` 中仅从 wheel 安装。

```
* Building wheel from sdist
* Getting build dependencies for wheel...
running egg_info
writing src\greetlab_25020007073.egg-info\PKG-INFO
writing dependency_links to src\greetlab_25020007073.egg-info\dependency_links.txt
writing entry points to src\greetlab_25020007073.egg-info\entry_points.txt
writing top-level names to src\greetlab_25020007073.egg-info\top_level.txt
reading manifest file 'src\greetlab_25020007073.egg-info\SOURCES.txt'
writing manifest file 'src\greetlab_25020007073.egg-info\SOURCES.txt'
* Building wheel...
running bdist_wheel
running build
running build_py
creating build\lib\greetlab
copying src\greetlab\cli.py -> build\lib\greetlab
copying src\greetlab\__init__.py -> build\lib\greetlab
running egg_info
writing src\greetlab_25020007073.egg-info\PKG-INFO
writing dependency_links to src\greetlab_25020007073.egg-info\dependency_links.txt
writing entry points to src\greetlab_25020007073.egg-info\entry_points.txt
writing top-level names to src\greetlab_25020007073.egg-info\top_level.txt
reading manifest file 'src\greetlab_25020007073.egg-info\SOURCES.txt'
writing manifest file 'src\greetlab_25020007073.egg-info\SOURCES.txt'
installing to build\bdist.win-amd64\wheel
running install
running install_lib
creating build\bdist.win-amd64\wheel
creating build\bdist.win-amd64\wheel\greetlab
copying build\lib\greetlab\cli.py -> build\bdist.win-amd64\wheel\greetlab
copying build\lib\greetlab\__init__.py -> build\bdist.win-amd64\wheel\greetlab
running install_egg_info
Copying src\greetlab_25020007073.egg-info to build\bdist.win-amd64\wheel\greetlab_25020007073-0.1.0-py3.13.egg-info
running install_scripts
creating build\bdist.win-amd64\wheel\greetlab_25020007073-0.1.0.dist-info\WHEEL
creating 'C:\msys64\ubuntuu-26.04\root\gitRepo\SystemToolKitDevCourse\week3\q09\dist\tmp-bwetkenz\greetlab_25020007073-0.1.0-py3-none-any.whl' and adding 'build\bdist.win-amd64\wheel' to it
adding 'greetlab/' to it
adding 'greetlab/cli.py'
adding 'greetlab_25020007073-0.1.0.dist-info\METADATA'
adding 'greetlab_25020007073-0.1.0.dist-info\WHEEL'
adding 'greetlab_25020007073-0.1.0.dist-info\entry_points.txt'
adding 'greetlab_25020007073-0.1.0.dist-info\top_level.txt'
adding 'greetlab_25020007073-0.1.0.dist-info\RECORD'
removing build\bdist.win-amd64\wheel
Successfully built greetlab_25020007073-0.1.0.tar.gz and greetlab_25020007073-0.1.0-py3-none-any.whl
```

图 1: 实验一构建过程: `python -m build` 从 sdist 构建 wheel, 成功生成 tar.gz 与 whl 安装包

4.3 结果

在 q09 之外运行 `sdt-greet --name 25020007073` 输出:

```
Hello, 25020007073!
```

确认 wheel 构建正确、入口点 `sdt-greet` 在干净环境可用。

5 课上实验二：让编程智能体进入可验证的修复循环

5.1 实验要求

复制 q09 为 q10, 添加失败测试: 当 `name` 只含空白字符时, `main` 应以 `SystemExit(2)` 结束。先运行测试确认失败, 再向编程智能体说明目标、约束和测试命令, 要求它修改实现并运行测试。人工检查 diff、撤销无关修改, 并在 `ai_log.md` 中记录。

5.2 过程

失败测试 (空白姓名应抛 `SystemExit(2)`) 先红后绿:

```
FAILED test_blank_name_exits_nonzero - Failed: DID NOT RAISE SystemExit
1 failed, 1 passed
```

智能体在 `parse_args()` 后增加两行：`if not a.name.strip(): raise SystemExit(2)`。人工 diff 确认仅涉及 `cli.py` 第 8-9 行，无无关修改；pytest 2 项全部通过。

5.3 结论

只要提供明确目标、约束与测试命令，智能体能完成最小化修复，但必须人工审查 diff 以确认未引入无关变更——这是”可验证修复循环”的关键。

6 课上实验三：把”无法处理”的协作材料改成可执行信息

6.1 实验要求

围绕”空姓名仍输出问候语”缺陷，改写三段质量较差的协作材料：Issue（”Windows 上运行不了，尽快修复”）、提交信息（”fix bug”）、评审意见（”这里写得不好，重写”）。

6.2 重写结果

Issue 重写为含环境、复现命令、期望/实际结果与待确认项的规范报告；提交信息使用祈使语气并说明”为什么”：

```
fix: reject whitespace-only name and exit with code 2

When --name contains only whitespace, the program previously printed
"Hello, !" and exited 0. Add validation in main() to detect blank
names, print an error message, and exit with SystemExit(2).
```

评审意见标注 Blocking 级别，指出具体行为、下游风险与建议动作。全文控制在 400 字以内。

7 课上实验四：修复一个可复现的线性回归训练循环

7.1 实验要求

补全 PyTorch 线性回归训练循环中的两个 TODO：清空梯度/反向传播/更新参数；进入评估模式并在 `no_grad` 中打印最终损失、`weight` 和 `bias`。最终损失应小于 0.001，不得直接把 `weight` 和 `bias` 赋值为 3 和 -1。

7.2 补全与结果

```
for _ in range(200):
    pred = model(x)
    loss = loss_fn(pred, y)
    opt.zero_grad()
    loss.backward()
    opt.step()
```

```
model.eval()
with torch.no_grad():
    final_loss = loss_fn(model(x), y)
    print(f"final_loss={final_loss.item():.6f}")
    print(f"weight={model.weight.item():.6f}")
    print(f"bias={model.bias.item():.6f}")
```

```
final_loss=0.000000
weight=2.999997
bias=-1.000000
```

```
(llm313) PS Microsoft.PowerShell.Core\FileSystem::\\wsl$\ubuntu-26.04\root\gitRepo\SystemToolkitDevCourse\week3\q12> python train.py
final_loss=0.000000
weight=2.999997
bias=-1.000000
```

图 2: 实验四运行结果: train.py 训练 200 轮后输出 final_loss、weight、bias

最终损失 $0.000000 < 0.001$, weight 接近 3、bias 接近 -1 , 与目标函数 $y = 3x - 1$ 一致; SGD (学习率 0.1) 在 200 轮收敛, 未直接赋值参数。

8 课后练习（第三周 · 共 12 个）

8.1 练习一: venv 环境差异——printenv 对比与 deactivate

要求: 创建 venv、激活、printenv 前后各存一个文件并 diff, 分析环境变化与 shell 偏好 venv 的原因; 运行 which deactivate 并解释该函数作用。

做法与结果: 创建 .venv 后 source activate, printenv 输出 before.txt/after.txt 并 diff。变化集中在三点: **PATH 前置遮蔽** (.venv/bin 插到 PATH 最前, 使 which python3 从 /usr/bin/python3 变为 .venv/bin/python3)、新增 VIRTUAL_ENV 环境变量、PS1 增加“(venv)”前缀。shell 偏好 venv 是因为 PATH 查找按顺序进行, 前置目录优先命中。deactivate 是 activate 脚本用 source 加载进当前 shell 的 **shell 函数**, 执行时从 PATH 去掉 venv 前缀、unset VIRTUAL_ENV、恢复 PS1; 只有函数能修改当前 shell 环境, 因此必须 source 而不能直接执行。

8.2 练习二: 创建 Python 包并生成 lockfile

要求: 创建带 pyproject.toml 的包并安装进 venv, 创建并检查 lockfile。

做法与结果: 用 uv 创建 msgreet 包 (pyproject.toml + src 布局 + [project.scripts] 入口), uv build 生成 sdist 与 wheel 并安装进 venv, 运行输出 “Hello, World!”。uv lock 生成 uv.lock, 其中每个依赖都记录 url、sha256 哈希、size 与 upload-time 等锁定信息; uv sync --frozen 从 lockfile 独立重建并运行成功, 证明 lockfile 自洽、可复现。

8.3 练习三: Docker 本地构建 Missing Semester 课程网站

要求: 安装 Docker, 用 docker compose 在本地构建 Missing Semester 官网。

做法与结果: 克隆官方仓库并编写 docker-compose.yml (Jekyll 站 + 代理两个 service)。适配本机 fake-IP DNS 问题: 容器内 apk 拉包需经宿主代理 (HTTP_PROXY=http://172.26.224.1:7890 build-args) 才能成功。最终容器 Up、端口 4000 映射, curl 返回 HTTP 200, 首页正确渲染 2026 课程内容, compose down 清理干净。

8.4 练习四：Dockerfile + docker-compose 运行应用与 Redis 缓存

要求：为 Python 应用写 Dockerfile，再写 docker-compose.yml 让应用与 Redis 一起运行。

做法与结果：Dockerfile 基于 python:3.12-slim 安装依赖并启动 Flask 应用（访问 / 使 Redis 计数递增）；docker-compose.yml 定义 web 与 redis:7-alpine 两个 service。验证跨容器计数连续递增（1, 2, 3, ...），redis-cli 读到同一计数，证明 web 与 redis 数据一致、容器网络正常。

8.5 练习五：发布包到 TestPyPI 并推送镜像到 ghcr.io

要求：发布 Python 包到 TestPyPI；构建含该包的 Docker 镜像并推到 ghcr.io。

做法与结果：构建 q05pkg-25020007073 包，twine check PASSED。因本机无公网凭据，先在机制完全一致的本地等效环境跑通全链路：pypiserver 等效 TestPyPI（twine upload + pip install -index-url 安装运行成功）、本地 registry 等效 ghcr.io（docker tag/push/pull 全通过）。随后在拿到 GitHub 令牌后完成**真实发布**：镜像已推送至 ghcr.io/mrllliao/q05pkg-25020007073:0.1.0（digest sha256:f644c18e...），可 pull 并运行，镜像内 CLI 输出“Hello, World!”。TestPyPI 已用独立 token 完成**真实上传**：wheel + sdist 发布于 <https://test.pypi.org/project/q05pkg-25020007073/0.1.0/>，全新 venv 中 pip install -index-url <https://test.pypi.org/simple/> 安装并运行 q05hello 输出“Hello, World!”（exit 0）。

8.6 练习六：用 GitHub Pages 做网站

要求：用 GitHub Pages 建网站（额外项：自定义域名）。

做法与结果：先本地用 git worktree 建 gh-pages 孤儿分支演示发布机制（gh-pages 只含站点文件，main 工作区不被污染），本地 curl 验证 HTTP 200、标题与学号正确渲染。随后完成**真实上线**：gh-pages 分支推送到 GitHub，Pages 配置为从 gh-pages 分支发布，站点地址 <https://mrllliao.github.io/SystemToolkitD> 访问返回 HTTP 200，标题“廖世嘉 · 系统开发工具基础”正常显示。

8.7 练习七：同一编程任务用四种模式各做一次

要求：把同一编程任务分别用手动编码、AI 自动补全、内联聊天、智能体各做一次并比较。

做法与结果：选择 wordcount 小功能，四种模式各实现一次。四种产物在 -top 4 输出上**逐字节一致**；针对 -l/-w/-c 三种统计标志做了列过滤修复；记录各模式在提示成本、迭代速度、人工介入程度上的体验差异（手动最可控但最慢，智能体最快但需审阅 diff）。

8.8 练习八：用 AI 智能体浏览陌生代码库

要求：用 AI 编程智能体浏览一个陌生代码库（本练习选择 tenacity 重试库）。

做法与结果：克隆 tenacity 后让智能体建立可核验心智模型，产出证据矩阵与“责任闭合卡”（每条结论标注入口与调用链证据）。对两条核心结论（重试策略组合与 enabled=False 短路行为）做真实运行验证，均 PASS，证明结论有据可查而非凭空猜测。

8.9 练习九：氛围编程（Vibe Coding）一个小应用

要求：从零开始氛围编程一个小应用，不手动写一行代码。

做法与结果：通过自然语言描述需求，生成一个”番茄钟 + 待办”单文件应用（零依赖 HTML）。自动验证：GET /index.html HTTP 200、文件约 8.5 KB、标题正确；结构自检确认倒计时默认 25:00、开始/暂停/重置、待办增删等关键功能齐全，无外部依赖。

8.10 练习十：为编程智能体创建 AGENTS.md、Skill 与子智能体

要求：创建并测试 AGENTS.md（或等效文件）、一个 Skill、一个子智能体，思考何时用哪一种。

做法与结果：在仓库根创建 AGENTS.md（记录项目结构、命令、分步提交规范与环境约束），后续所有提交都遵循它。创建 todo-audit Skill（扫描 TODO/FIXME/HACK 标记，支持 CI 退出码）：样例精确检出 3 处，仓库扫描发现 21 处匹配（含 q09 的 TODO_KEY 子串误报），并因首次扫描把 .venv 第三方库 1003 处标记计入而**测试驱动打补丁**（排除依赖/构建目录）。创建 homework-reviewer 子智能体定义，其审查脚本如实发现 q05 的 solve.sh 未用 dirname 约定，证明子智能体不是橡皮图章。结论：AGENTS.md 管”规矩”、Skill 管”本事”、子智能体管”专职角色”。

8.11 练习十一：Markdown 无序列表正则（不直接编辑文件）

要求：用智能体完成与代码质量一讲中 Markdown 无序列表正则练习相同的目标，但**不通过直接编辑文件完成**（提示：用第一讲的命令行工具）。

做法与结果：构建只读命令行流水线（grep -P / sed -E / awk / md5sum）：朴素正则 `^\s*[-*+]\s+` 命中 9 处（含代码块内 2 行误报）；awk 维护三反引号围栏状态机后**精确匹配 7 处**；负例（有序列表、水平线 —）匹配数均为 0；md5sum 前后一致证明源文件零改动。结论：正则只能看到”行”看不到”代码块”上下文，需状态机；智能体直接编辑文件有不可审阅、难回滚、易误伤等缺点，改为”生成 + 审阅”的流水线更安全。

8.12 练习十二：搭建隔离环境运行 YOLO 模式智能体

要求：YOLO 模式（如 `-dangerously-skip-permissions`）直接用在宿主上不安全，在虚拟机/容器等隔离环境中启用才可接受；搭建这样的环境（参考 Claude Code devcontainers / Docker Sandboxes 思路）。

做法与结果：编写 Dockerfile（非 root 用户 agent）+ docker-compose.yml，将沙箱配置为：read_only 根文件系统、cap_drop ALL（丢弃全部 Linux 能力）、no-new-privileges、pids_limit 64、mem_limit 256m、network_mode none、命名卷 workdir（唯一可写盘区）。在沙箱内运行边界验证，8 项全部通过：CapEff=0000000000000000（无能力）、NoNewPrivs=1（不能提权）、根挂载为 overlay ro、写 /etc 被拒、接口只剩 lo 且 curl 失败（断网）、pids.max=64、memory.max=268435456、/work/workdir 可正常写盘。由此证明：即便智能体在自主模式下”犯浑”，最坏也只是毁掉一个可丢弃的容器，宿主毫发无损。

9 GitHub 提交记录

按照考核要求，本周所有实验与课后练习均已上传到个人 GitHub 仓库，遵守”分步提交、禁止单次全量提交”规范：12 个课后练习各一个 commit，外加 README 文档提交，共 13 个提交；全仓库累计 81 个提交，工作区干净，全部已推送至远端。


```

廖世嘉 SystemToolkitDevCourse commit 记录 (2026-08-31 第3周课后练习)

zhouliao@wsl: ~/SystemToolkitDevCourse$
git log --graph --oneline -13

1826488 docs: README补充week3 homework q01-q12说明
f2eabcb feat(homework-q12): 搭建容器隔离沙箱供YOLO模式智能体运行(cap_drop/read_only/断网/pids/mem限制,8项边界验证通过)
1927f4b feat(homework-q11): 用grep/sed/awk只读流水线完成Markdown无序列表正则练习(awk围栏状态机精确匹配7处,源文件md5零改动)
315f82c feat(homework-q10): 为编程智能体创建并测试AGENTS.md(仓库根)、todo-audit skill(含venv排除补丁)、homework-reviewer子智能体(如发现q05未用dirname)
1032a5c feat(homework-q09): 氛围编程一个番茄钟+待办单文件应用(零依赖),全程不手写代码,起站+结构/功能自检通过
9eb6848 feat(homework-q08): 用智能体浏览陌生代码库tenacity,产出证据矩阵/责任闭卡/真实运行验证(策略组合+enabled=False)
a7b95a8 feat(homework-q07): 同一wordcount小功能用手动/自动补全/内联聊天/智能体四种模式各做一次,产物行为逐字节一致,记录体验差异并修复-l/-w/-c列过滤
97ba8f0 feat(homework-q06): 用GitHub Pages建个人静态网站,本地curl验证HTTP 200,git worktree孤儿分支演示gh-pages发布机制(真实push需凭据)
5c89186 feat(homework-q05): 构建q05pkg并演示发布链路(本地pypiserver等效TestPyPI+本地registry等效ghcr.io),twine upload/install/镜像push/pull全通过
847eaf3 feat(homework-q04): 编写Python应用Dockerfile与docker-compose.yml(web+redis),验证跨容器计数递增与Redis数据一致
9e8dd32 feat(homework-q03): 用docker compose本地构建missing-semester网站,适配本机网络(fake-IP DNS下apk经宿主代理build-args成功),curl验证HTTP 200
3e5694f feat(homework-q02): 创建msgreet包(pyproject.toml+src布局)安装进venv,用uv lock生成lockfile并检查sha256/URL,uv sync --frozen验证可复现
d0871ee feat(homework-q01): venv激活前后printenv对比,分析PATH前置遮蔽与VIRTUAL_ENV/PS1变化,type deactivate还原函数机制

83 commits on main pushed to github.com/MrLLiao/SystemToolkitDevCourse

```

仓库地址: <https://github.com/MrLLiao/SystemToolkitDevCourse>

本周课后练习提交记录 (commit 摘要):

表 2: 本周课后练习提交记录

哈希	类型	提交说明
d0871ee	feat	homework-q01: venv/printenv 对比与 deactivate 原理
3e5694f	feat	homework-q02: 创建 msgreet 包 + pyproject.toml + lockfile
9e8dd32	feat	homework-q03: Docker 构建 missing-semester 网站, HTTP 200
847eaf3	feat	homework-q04: Dockerfile + compose + Redis 缓存, 计数一致
5c89186	feat	homework-q05: 发布链路演示 (pypiserver/registry 等效)
97ba8f0	feat	homework-q06: GitHub Pages 静态站 + gh-pages 发布机制
a7b95a8	feat	homework-q07: 四种编码模式对比, 产物逐字节一致
9eb6848	feat	homework-q08: 智能体浏览 tenacity, 证据矩阵 + 真实验证
1032a5c	feat	homework-q09: 氛围编程番茄钟 + 待办应用 (零手写代码)
315f82c	feat	homework-q10: AGENTS.md + todo-audit Skill + 子智能体
1927f4b	feat	homework-q11: 无序列列表正则 (命令行只读完成, md5 零改动)
f2eabcb	feat	homework-q12: YOLO 模式容器隔离沙箱, 8 项边界通过
1826488	docs	README 补充 week3 homework q01-q12 说明

10 实验结果汇总

表 3: 本周实验与练习完成情况

编号	内容	验证结果
课上实验	构建 Wheel / 智能体修复 / 协作改写 / 线性回归	全部通过, 损失 < 0.001
练习 1	venv 环境对比	diff 定位 PATH 遮蔽与 VIRTUAL_ENV/PS1 变化
练习 2	包 + lockfile	uv sync --frozen 独立重建成功
练习 3	Docker 构建网站	容器 Up, HTTP 200
练习 4	Docker + Redis	计数跨容器递增且一致
练习 5	TestPyPI / ghcr.io	本地全链路通过; ghcr.io 真实推送 + TestPyPI 真实上传成功
练习 6	GitHub Pages	站点真实上线, HTTP 200
练习 7	四种编码模式	产物逐字节一致
练习 8	智能体浏览代码库	两条结论被真实运行证实
练习 9	氛围编程	功能自检全通过, 零依赖
练习 10	AGENTS/Skill/子智能体	skill 检出真实标记, 子智能体发现真问题
练习 11	无序列表正则	awk 精确 7 处, 负例全 0, md5 零改动
练习 12	YOLO 隔离沙箱	8 项边界全部通过

11 问题与解决方法

11.1 本机 fake-IP DNS 与代理问题 (练习三/四/十二)

本机 WSL 存在 fake-IP DNS 问题, 容器内 apt/apk 拉包与 git 访问 github.com 都会失败。解决: git 全局代理设为宿主代理 `http://172.26.224.1:7890`; Docker 构建传入 `HTTP_PROXY/HTTPS_PROXY` build-args。此问题在练习三中首次定位并记录。

11.2 真实发布需要凭据 (练习五/六)

本机默认无任何公网凭据。练习五先以机制一致的本地等效环境 (pypiserver/本地 registry) 跑通全链路; 在用户提供 GitHub 令牌后完成 ghcr.io 真实推送与 GitHub Pages 真实上线。TestPyPI 需要 test.pypi.org 单独签发的 API token (GitHub 令牌、pypi.org 令牌都不能用于 test.pypi.org), 拿到专属 token 后已完成真实上传。

11.3 安全策略拦截”演示破坏性命令” (练习十二)

练习十二最初的演示脚本本想”真的尝试 `rm -rf /etc、mount`”等越界命令, 被系统安全策略拦截。改为只读探测边界的方式: 直接读取 CapEff/NoNewPrivs/挂载/cgroup 证明配置生效, 效果等价且更安全。

11.4 智能体直接编辑文件的局限（练习十一）

智能体直接编辑文件不可审阅、难回滚、易误伤、不能沉淀复用。解决：用第一讲的命令行工具（grep/sed/awk）构建只读流水线，源文件 md5 前后一致作为”未改动”的证据。

12 解题感悟与实验总结

本周最大的收获是建立”可复现 + 可验证”的工程习惯。每个练习都配有 solve.sh（可重跑）与 output.txt（留证），并用 git 分步提交保证可追溯；12 个练习全部完成、全部提交、工作区干净并已推送远端。

在智能体编程方面，我体验到四种编码模式的差异：手动编码可控但慢，自动补全效率高但只适合局部，内联聊天适合边聊边改，智能体适合端到端任务但必须审阅 diff。AGENTS.md、Skill、子智能体三种机制分别回答”规矩、本事、角色”的问题，可以叠加使用。

在工程化方面，Docker 与发布链路让我理解了”本地等效 + 真实上线”的分步验证策略；隔离沙箱实验让我认识到自主模式的边界必须由基础设施兜底（无能力、只读根、断网、进程/内存受限），而不是依赖智能体”自觉”。正则练习则再次印证：工具链（命令行流水线）比”直接改文件”更安全、可复现、可组合。